



# HE1025 Operativsystem

F2: Virtualisering Processorn Policyer



## Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

**F2: V#3 Schemaläggning**

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: V#6

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 Buffer

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

**Kap 7..9,(10-11), RTOS**

Lab #1 Handledning

Kap 12..13, 15..17

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

t.b.d

Kap (47, 39) & 48, (49..51)

Lab #4 Handledning

**TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!**

**LAB1: Närvaro L1..4**



**Begrepp**

Process

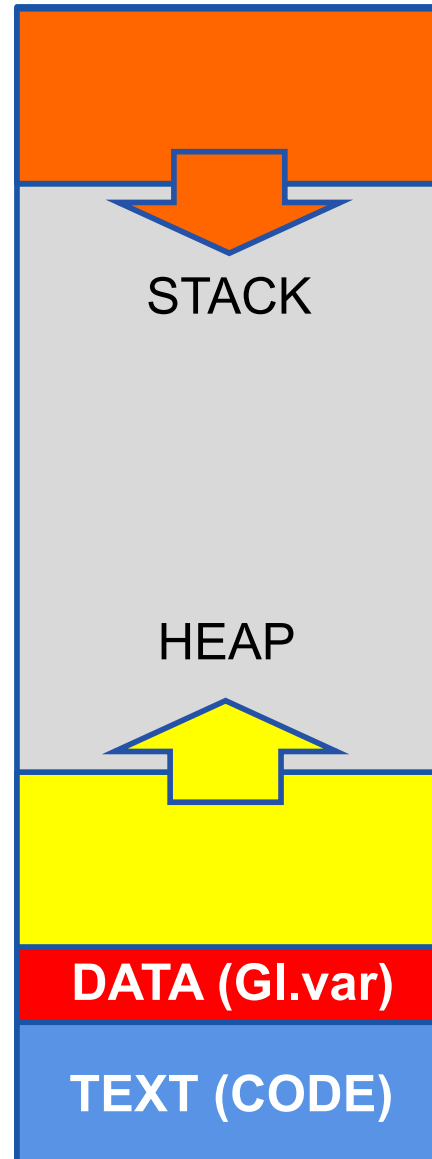
Machine State

**Scheduler**

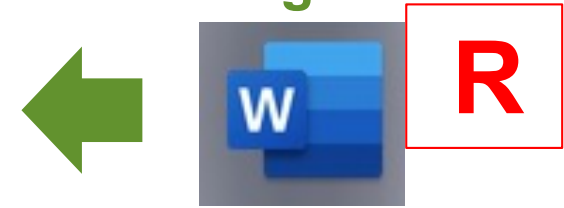
Context Switch

Virtualization

Process



Program



Memory

**Mem. Us.**

Virtualization





# Begrepp

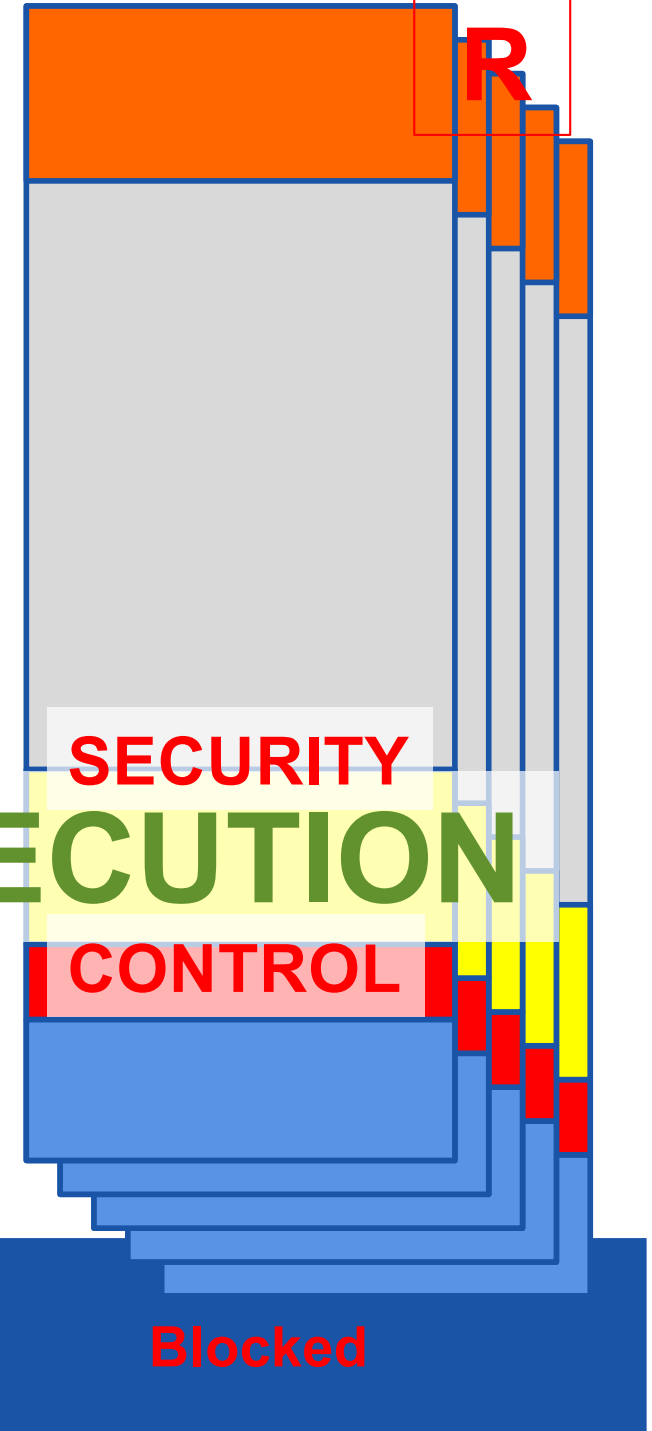
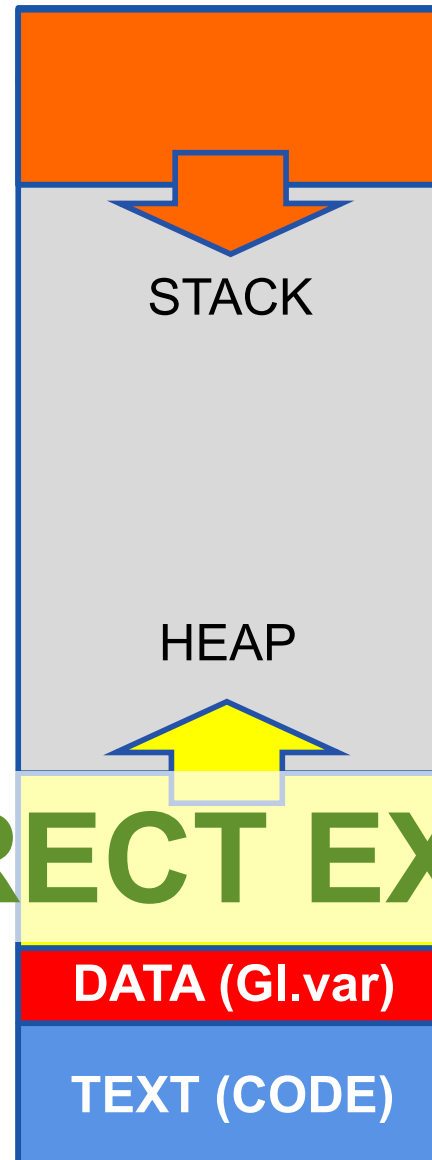
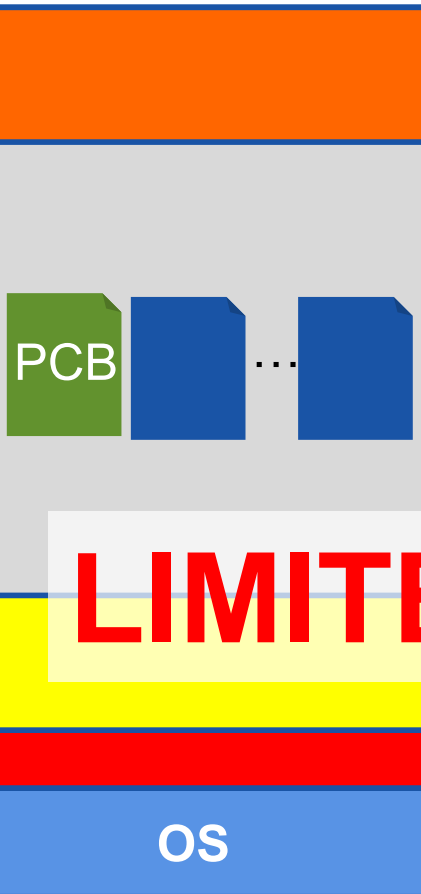
**RUNNING**

**READY**

**R**

SUPERVISOR MODE

USER MODE



**LIMITED DIRECT EXECUTION**

Process Control Block



**Blocked**



**OS @ boot**  
**(kernel mode)**  
**initialize trap table**

**Hardware**

remember address of...  
syscall handler

**Interrupt  
Descriptor  
Table**

256 entries  
0..x1F OS  
x80 trap

**OS @ run**  
**(kernel mode)**

**Hardware**

**Program**  
**(user mode)**

Create entry for process list  
Allocate memory for program  
Load program into memory  
Setup user stack with argv  
Fill kernel stack with reg/PC  
**return-from-trap**

restore regs from kernel stack  
move to user mode  
jump to main

Run main()

...  
Call system call  
**trap** into OS



eax=SC#  
syscall/  
int 0x80

save regs to kernel stack  
move to kernel mode  
jump to trap handler

Handle trap  
Do work of syscall  
**return-from-trap**

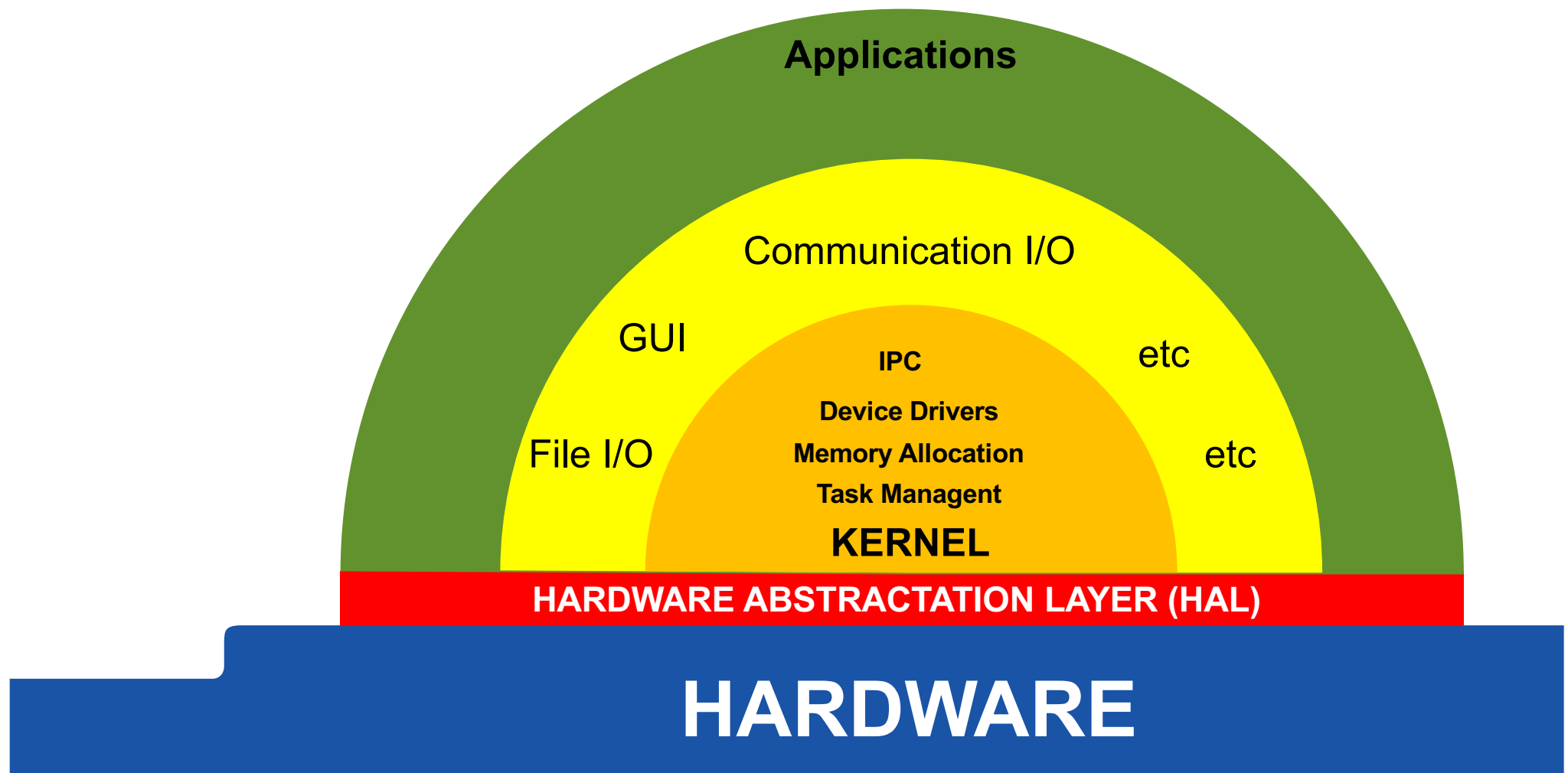
restore regs from kernel stack  
move to user mode  
jump to PC after trap

...  
return from main  
**trap** (via `exit()`)

sysret

Free memory of process  
Remove from process list

# Galet complex programvara – lager!



## F2: Scheduling + (FREE) RTOS...

NYTT!



# Troughput vs Fairness



## Metrics: Turnaround time...

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

**Assume we have a set of jobs...**

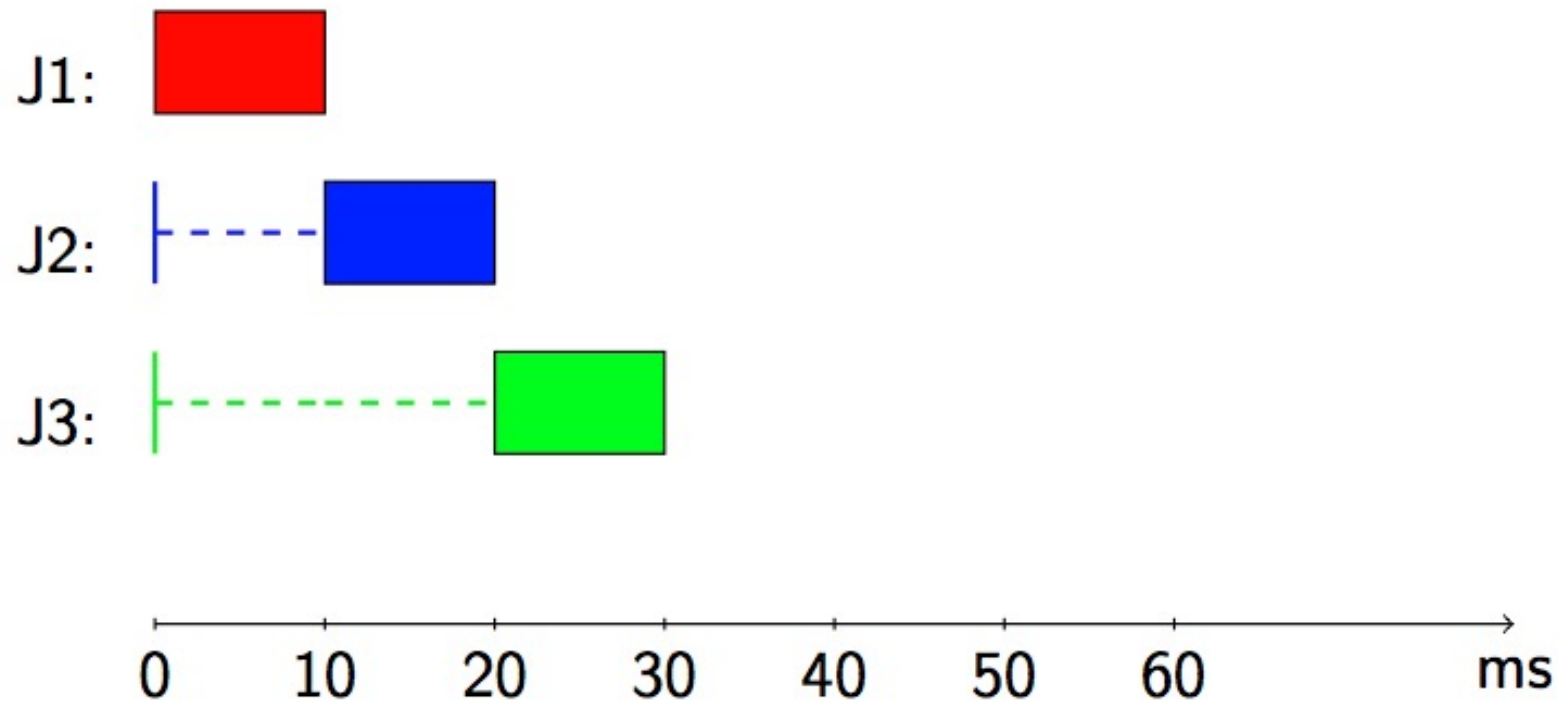
- 1. Each job takes an equal amount of time.**
- 2. All jobs arrive at the same time.**
- 3. A job will run to completion.**
- 4. The jobs only use the CPU (no I/O etc).**
- 5. The run-time of each job is known.**

**...this is unrealistic - we will relax these requirements!**



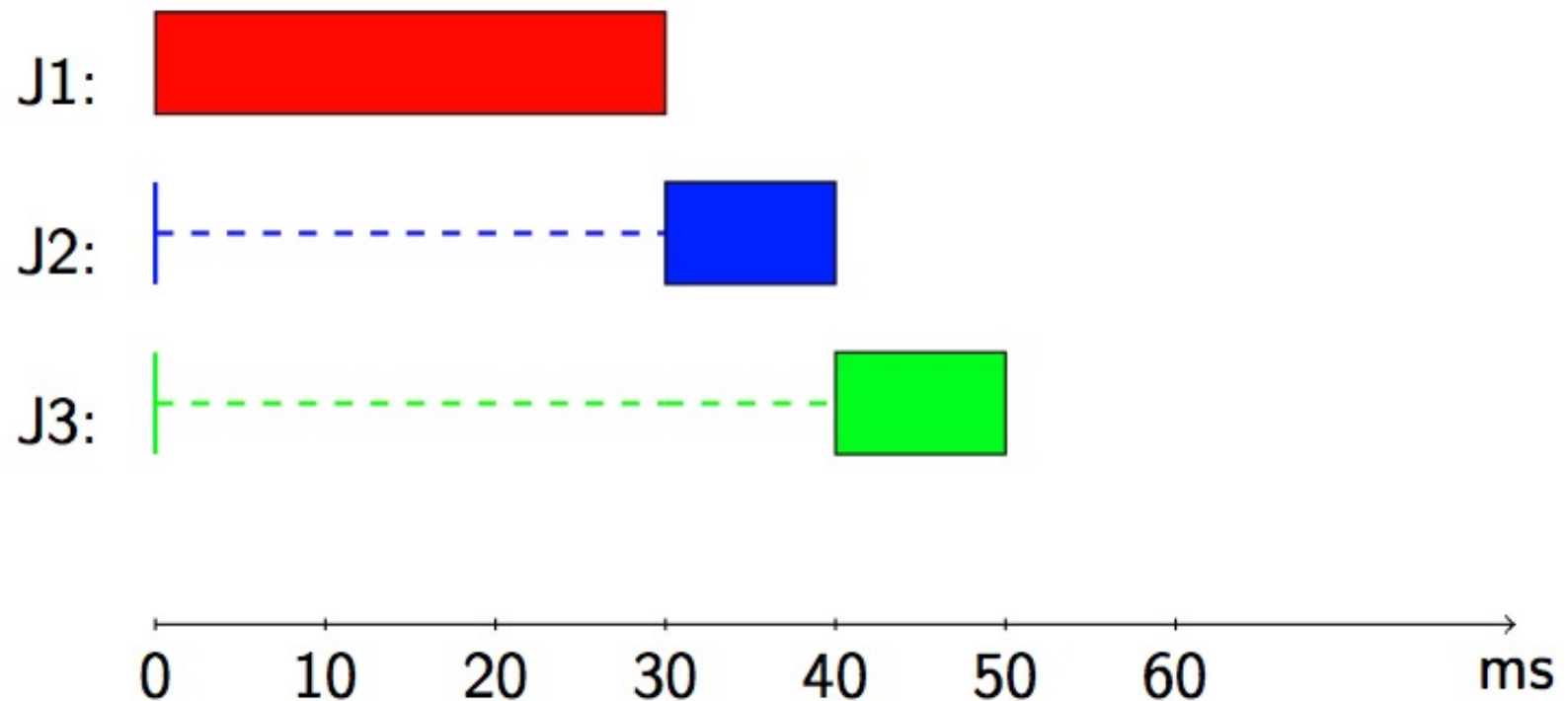


## First Come, First Served (FCFS aka FIFO)



$$T_{\text{att}} = (10 + 20 + 30) / 3 = 20\text{ms!}$$

## Alltid en bra taktik?

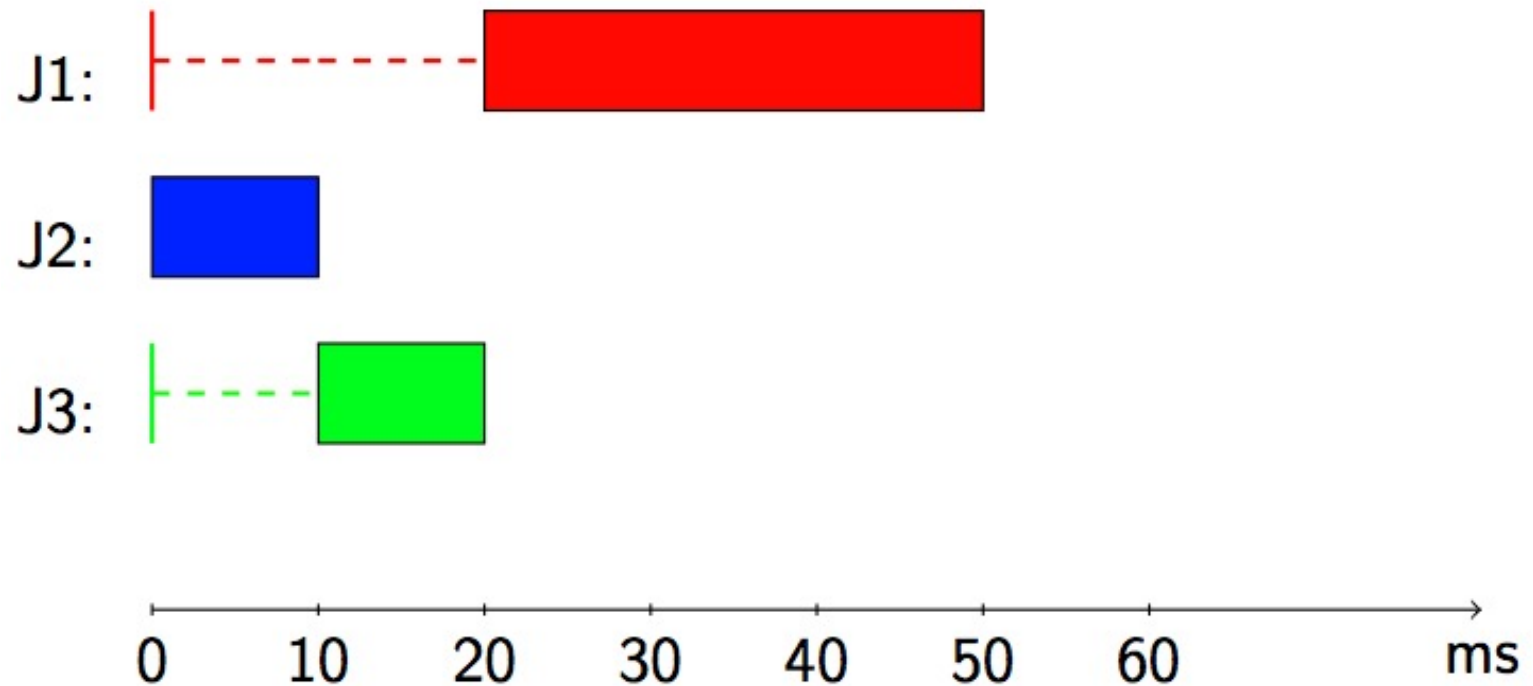


$$T_{\text{att}} = (30 + 40 + 50) / 3 = 40\text{ms!}$$

NEJ  
Kö...

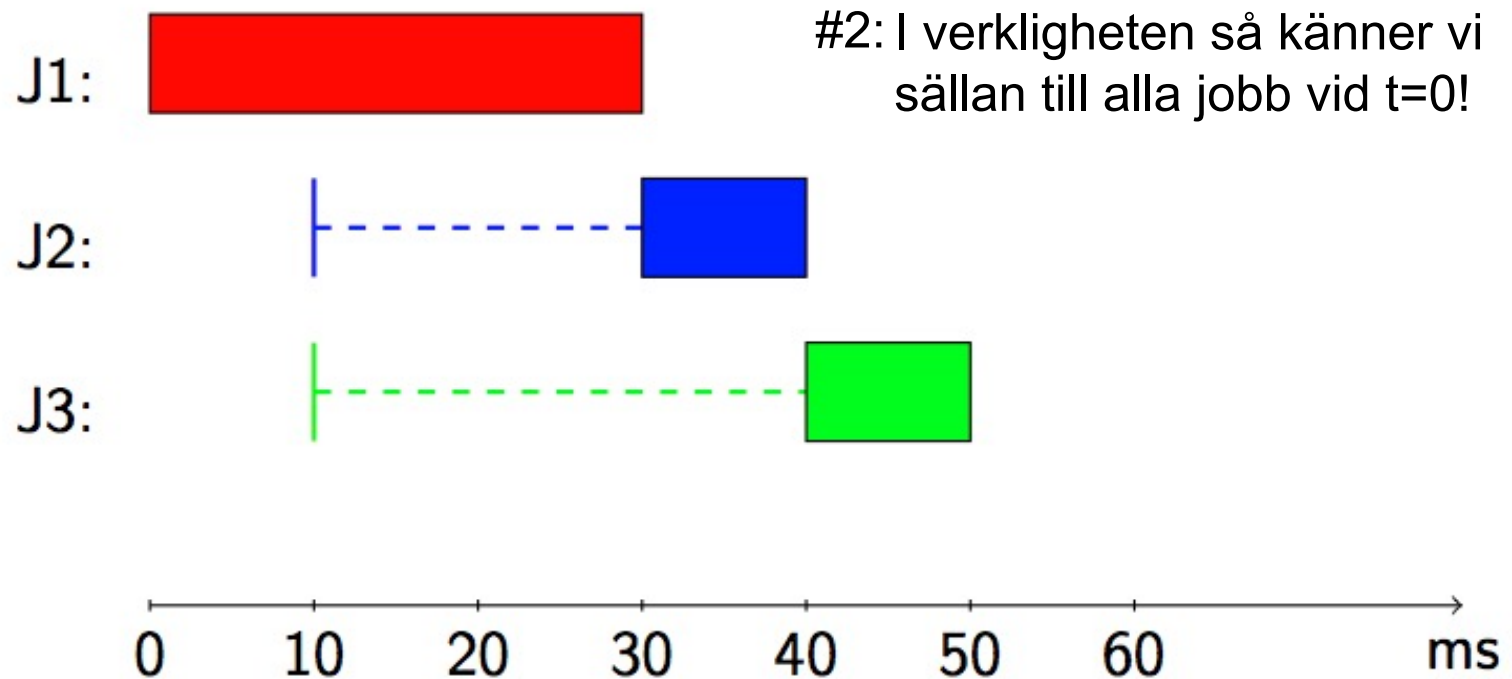


## Shortest Job First (SJF)



$$T_{\text{att}} = (10 + 20 + 50) / 3 = 27\text{ms!}$$

## Alltid en bra taktik?

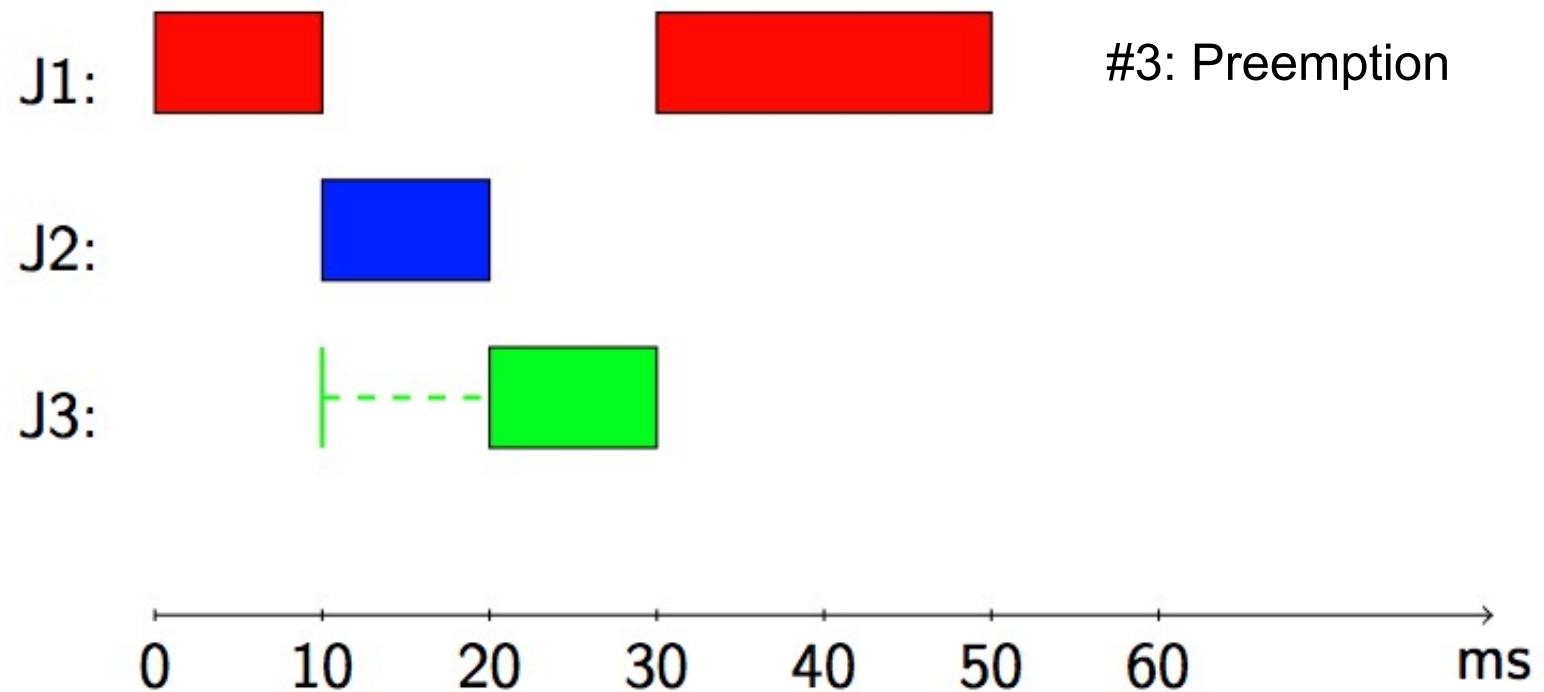


$$T_{\text{att}} = (30 + 40 + 50) / 3 = 40\text{ms!}$$

Nej!



# Shortest Time to Completion First (STCF) (aka PSJF)

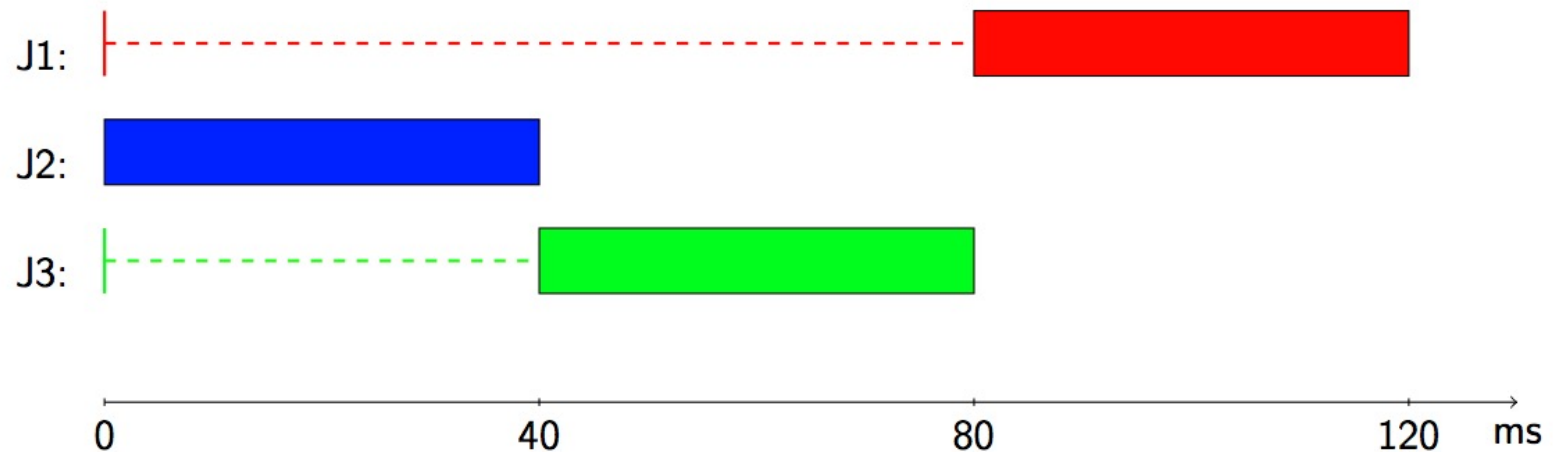


$$T_{att} = ((50-0) + (20-10) + (30-10)) / 3 = 27\text{ms!}$$



## New metrics: Response time

$$T_{\text{response}} = T_{\text{first\_run}} - T_{\text{arrival}}$$

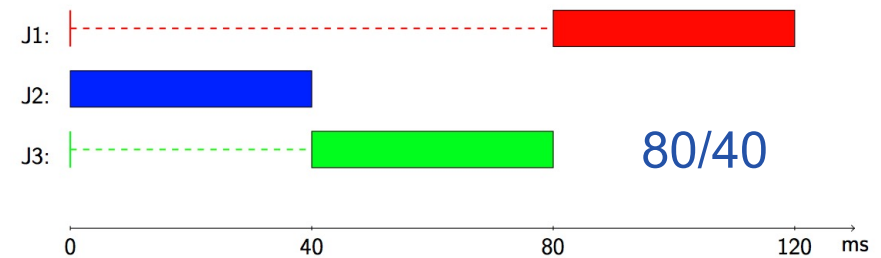


$$(T_{\text{att}} = (40 + 80 + 120) / 3 = 80\text{ms})$$

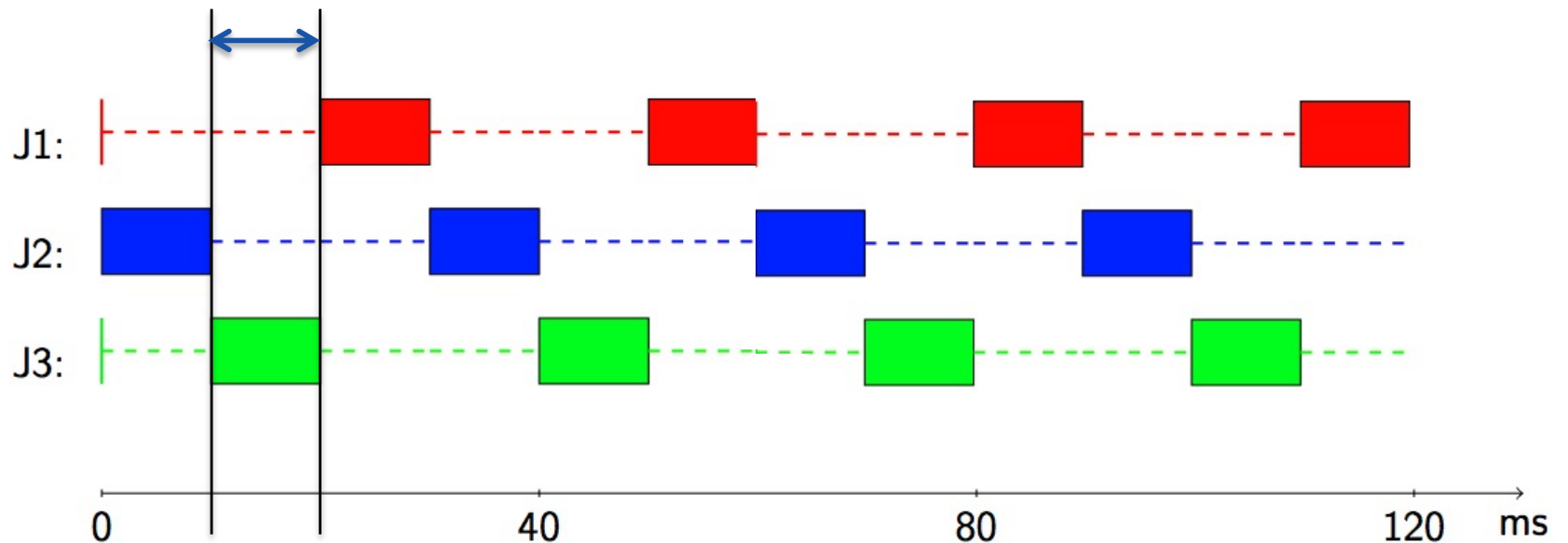
$$T_{\text{art}} = (0 + 40 + 80) / 3 = 40\text{ms!}$$



# Round Robin (RR)



Time slice/quantum

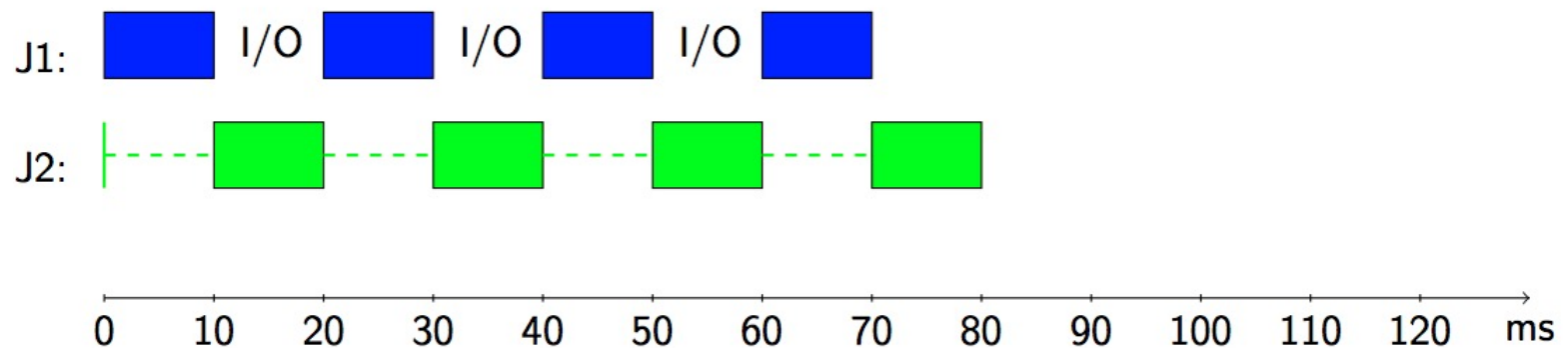
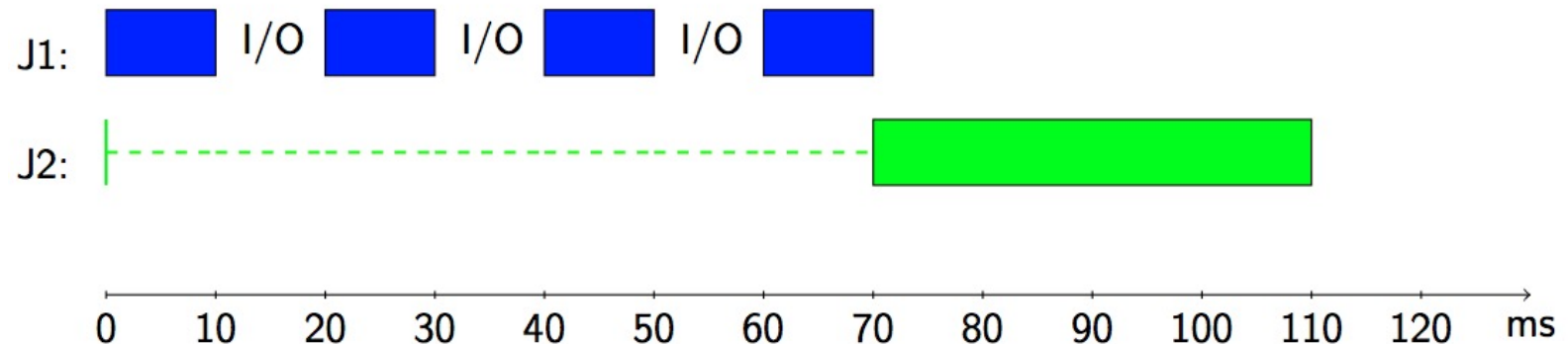


$$(T_{att} = (100 + 110 + 120) / 3 = 110\text{ms})$$

$$T_{art} = (0 + 10 + 20) / 3 = 10\text{ms!}$$

Starvation free  
Slice short vs  
Cost C.Switch

## Incorporating I/O (limitation #4)...



...i verkligheten vet vi sällan hur lång tid ett jobb tar (krux #5)





**Med andra ord:**

Går det att skapa en schema-läggnings policy som ger en bra kompromiss mellan responstid och turn around time?



# Multi-level Feedback Queue (MLFQ)

## Goals:

- Good turnaround time - schedule jobs so that jobs with short time to completion are not delayed too much.
- Improve responsiveness of interactive jobs - schedule interactive processes more often.

## Idea:

- Multiple levels of priority - interactive jobs have higher priority.
- Each level uses round-robin to give processes an equal share. Processes can be moved to a higher or lower level depending on their behavior.



# Multi-level Feedback Queue (MLFQ)

## Basic rules:

- Rule 1: if  $\text{Priority}(A) > \text{Priority}(B)$  then A is scheduled for execution.
- Rule 2: if  $\text{Priority}(A) = \text{Priority}(B)$  then A and B are scheduled in round-robin.
- Rule 3: when a new job is created it starts with the highest priority.
- Rule 4a: a job that has to be preempted (time-slice consumed) is moved to a lower priority.
- Rule 4b: a job that initiates a I/O-operation (or yields) remains on the same level.

# Multi-level Feedback Queue (MLFQ)

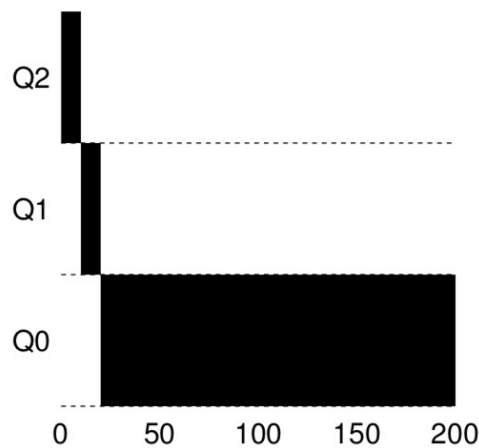


Figure 8.2: Long-running Job Over Time

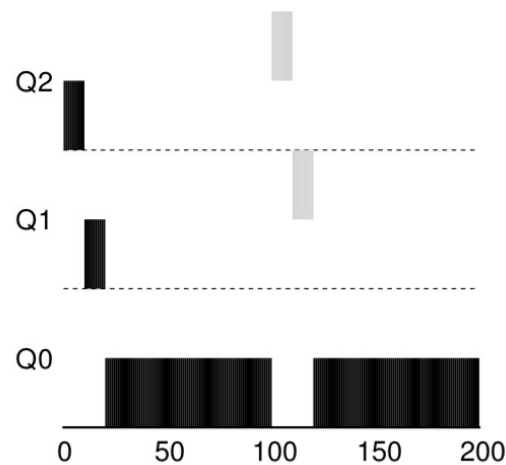
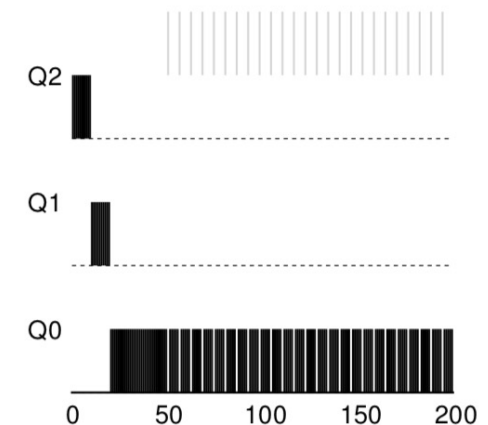


Figure 8.3: Along Came An Interactive Job



Hyfsat, men "tunga" jobb kan fördröjas oproportionerligt länge...  
...ett "listigt" "tungt jobb" kan också försöka luras!

# Multi-level Feedback Queue (MLFQ)

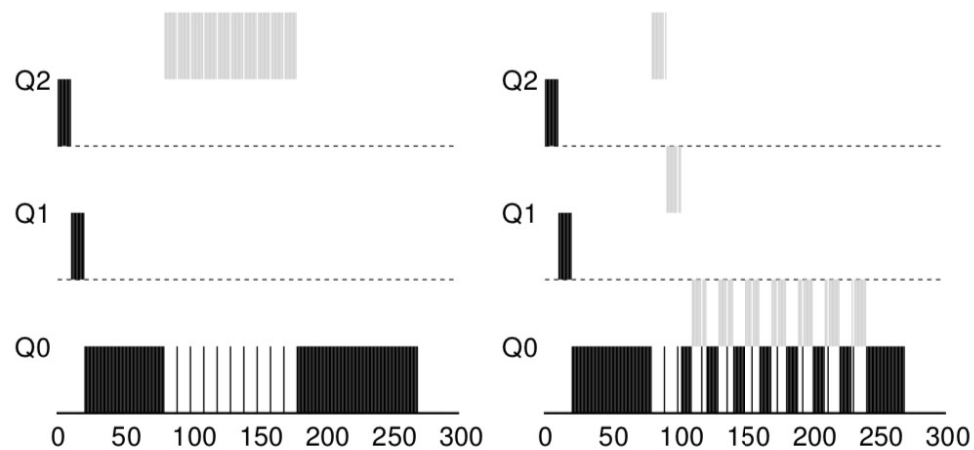


Figure 8.6: Without (Left) and With (Right) Gaming Tolerance

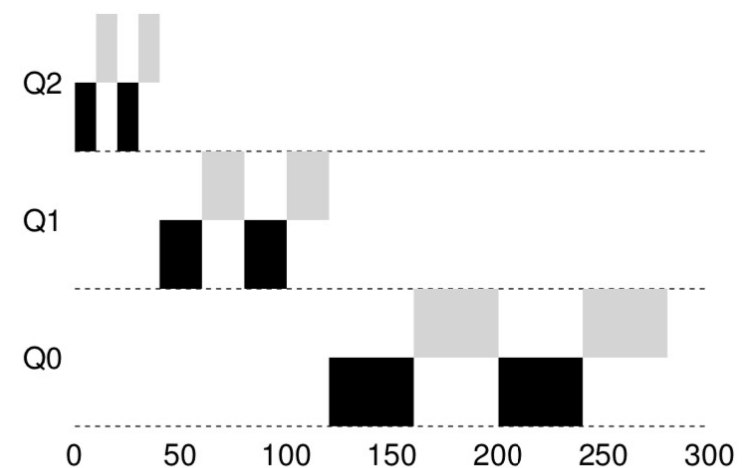


Figure 8.7: Lower Priority, Longer Quanta

Starvation, game the scheduler, Quanta+boost...



# Multi-level Feedback Queue (MLFQ)

- **Rule 1:** If  $\text{Priority}(A) > \text{Priority}(B)$ , A runs (B doesn't).
- **Rule 2:** If  $\text{Priority}(A) = \text{Priority}(B)$ , A & B run in round-robin fashion using the time slice (quantum length) of the given queue.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period  $S$ , move all the jobs in the system to the topmost queue.

However several attributes that need tuning: Time slice duration, number of queues, max allotted time(/queue?) & interval to boost?



## Ett alternativt angreppssätt...

MLFQ fungerar alltid men ger inte individuell kontroll över exekveringen av olika processer. Processer med likartat exekveringsbeteende hanteras på samma sätt...

...men någon gång vill användaren kanske kunna styra att en cpu-intensiv process får mycket cpu-tid och en annan gång att en I/O intensiv process faktisk kan bli lite slö...

...synsättet kräver att systemoperatören verkligen vet vad hen vill men då går det att skapa en modell som fungerar på följande sätt:



## Proportionerlig andel (Proportional Share)

En schemaläggare som bygger på ett lotteri...

...säg att det finns 100 lotter...

...process A äger 75 lotter, lott nummer 0..74 &

...process B äger 25 lotter, lott nummer 75..99!

När en process ska schemaläggas slumpar helt enkelt schemaläggaren ett tal mellan 0 och 99, processen som äger lotten schemaläggs!

Det här fungerar bäst om både process A & B ska schemaläggas många gånger...

(Schemaläggaren behöver bara vet hur många lotter det finns! En process kan få/ge lotter till en annan. Lokala lotterier & inflation!)





# Steg-schemaläggning (Stride scheduling)

Går det att hålla reda på två tal per process så kan vi till och med hoppa över lotteriet...

Skapa en konstant per process som kallas **steg(stride)** som är omvänt proportionerlig mot prioriteten samt en variabel **tur(pass)** som initialt är 0 men ökas med **steg(stride)** varje gång processen får köra!

Varje gång som schemaläggaren ska schemalägga en ny process så väljs helt enkelt den med lägst **tur(pass)** värde!



# Stride scheduling

| Pass(A)<br>(stride=100) | Pass(B)<br>(stride=200) | Pass(C)<br>(stride=40) | Who Runs? |
|-------------------------|-------------------------|------------------------|-----------|
| 0                       | 0                       | 0                      | A         |
| 100                     | 0                       | 0                      | B         |
| 100                     | 200                     | 0                      | C         |
| 100                     | 200                     | 40                     | C         |
| 100                     | 200                     | 80                     | C         |
| 100                     | 200                     | 120                    | A         |
| 200                     | 200                     | 120                    | C         |
| 200                     | 200                     | 160                    | C         |
| 200                     | 200                     | 200                    | ...       |

(Linux använde MLFQ upp till v2.6, i dag är CFS standard!)

Completely Fair Scheduler



# Completely Fair Scheduler (CFG/Linux)

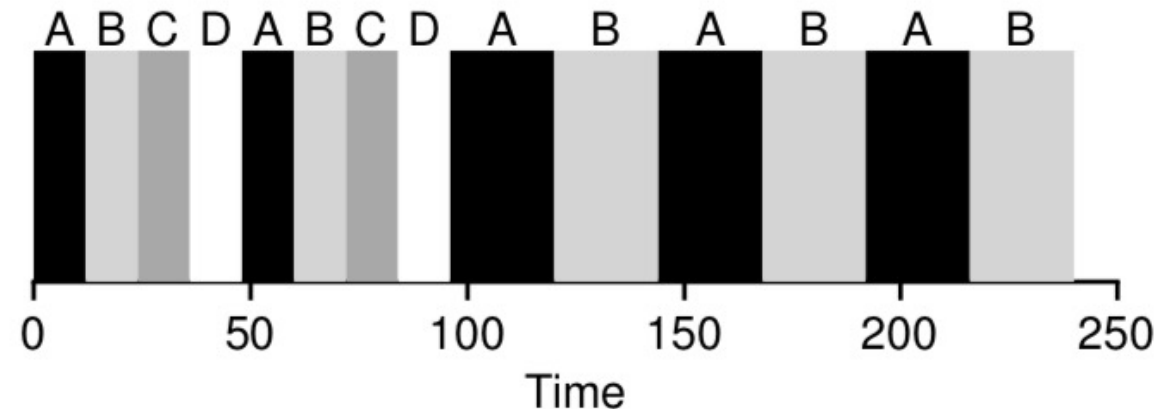


Figure 9.4: CFS Simple Example

Virtual runtime = accumulerad processortid

Sched\_latency(typ 48ms)/nb\_of\_processes = time slice

Min\_granularity(typ 6ms)

Priority (-20..+19) 
$$\text{time\_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched\_latency}$$

Välj den process som har lägst virtual runtime...  
...kör den en time slice (korrigerat för min\_granularity och priority)!



# Real-time systems

In real time scheduling we introduce a new requirement:  
*things should be completed within a given time period.*

- **Hard** : all deadlines should be met, missing a deadline is a failure.
- **Soft** : deadlines could be missed but the application should be notified and be able to take actions.

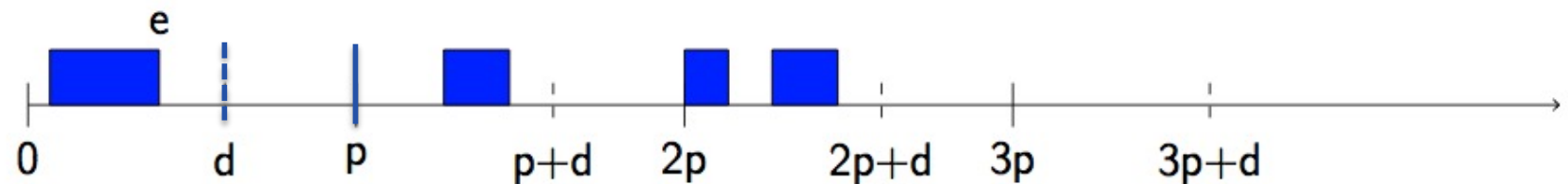


# Hard real-time systems

(ej i boken!)

In hard real-time systems, tasks are known in forehand and described by a triplet  $\langle e, d, p \rangle$

- $e$ : the worst case execution time for the task.
- $d$ : the deadline, when in the future do we need to finish.
- $p$ : the period, how often should the task be scheduled.



$$e \leq d \leq p$$



# Hard real-time systems

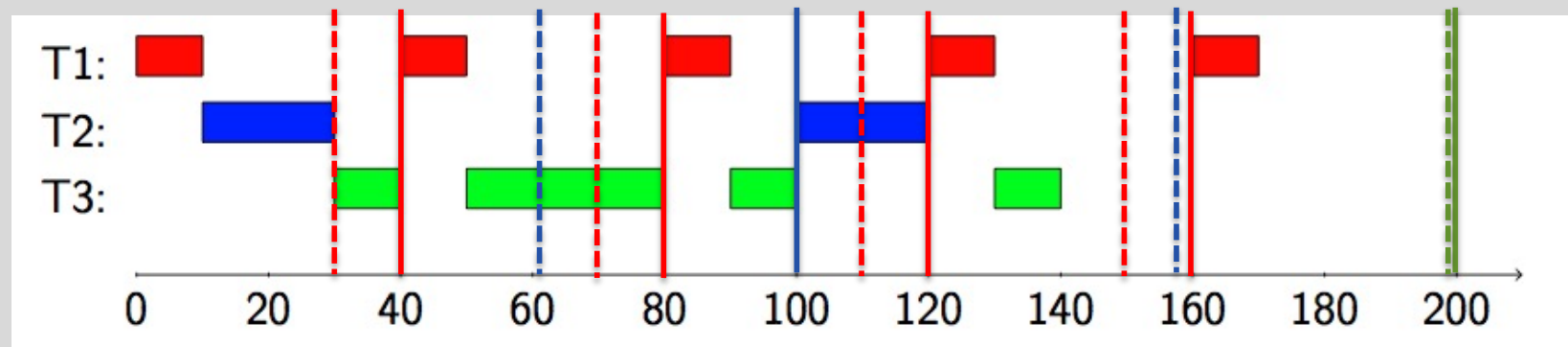
(ej i boken!)

Given a set of tasks:

T1:  $\langle 10, 30, 40 \rangle$ ,

T2:  $\langle 20, 60, 100 \rangle$ ,

T3:  $\langle 60, 200, 200 \rangle$ : , find the scheduling.



$$e < d \leq p$$



# Hard real-time systems

(ej i boken!)

Given that  $p = d$  i.e. a task must be completed within its period.

- **Rate Monotonic Scheduling (RMS):**
  - Schedule the available task with the shortest period
  - Always works if utilization is  $< 69\%$  (actually less than  $n * (2^{1/n} - 1)$ ), could work for higher loads.
  - Simpler to reason about, easy to implement.
- **Earliest Deadline First (EDF):**
  - Schedule based on the deadline, more freedom to choose tasks. Always works if utilization is  $< 100\%$ .
  - Used by Linux in the real-time extension (not in the regular system)

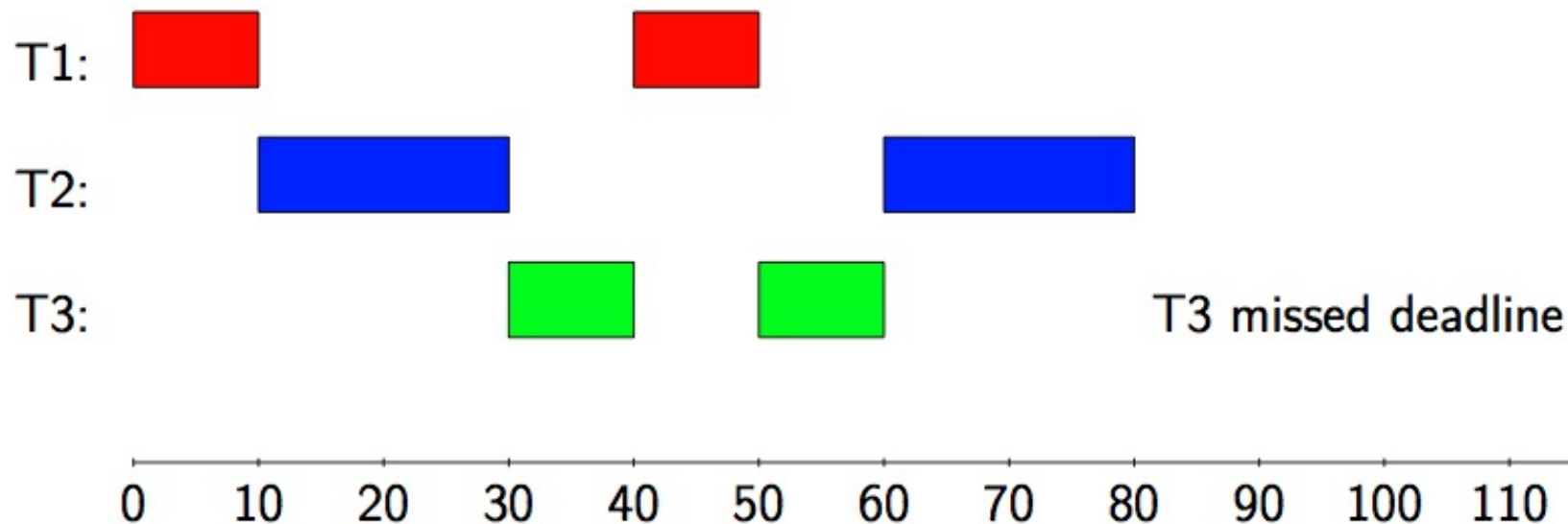


## Överkurs/räkna

# Rate Monotonic Scheduling (RMS)

Assume we have tasks: T1:  $\langle 10, 40, 40 \rangle$ , T2:  $\langle 20, 60, 60 \rangle$ , T3:  $\langle 30, 80, 80 \rangle$ .

$$10/40 + 20/60 + 30/80 = 6/24 + 8/24 + 9/24 = 23/24$$



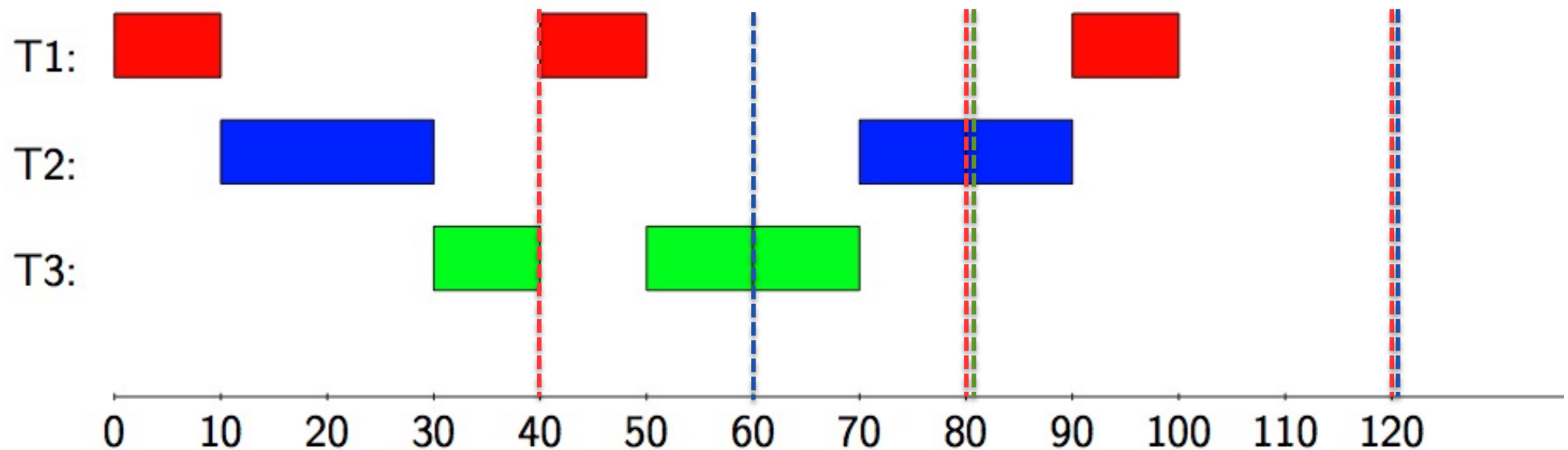




# Överkurs/räkna

Assume we have tasks: T1:  $\langle 10, 40, 40 \rangle$ , T2:  $\langle 20, 60, 60 \rangle$ , T3:  $\langle 30, 80, 80 \rangle$ .

$$10/40 + 20/60 + 30/80 = 6/24 + 8/24 + 9/24 = 23/24$$





# FreeRTOS



Har utvecklats av ledande aktörer allt sedan millennieskiftet...  
...överlägset störts på marknaden, laddas ner var annan min!

Allmänt ger ett RTOS:

- Abstraherar bort timing information
- Underlättar utveckling, testning & underhåll
- Prestanda & Power management (idle/int)

Specifikt FreeRTOS därutöver:

- Beprövad, gratis, minimal kodbas (3 filer!)
- Omfattande developer community



# FreeRTOS



```
while (1) {  
    idle++;           // Manage Performance Monitoring!  
    LCD_WR_Queue();   // Manage LCD com queue!  
    U0_TX_Queue();     // Manage U(S)ART TX Queue!  
    ADC0_Management(); // Manage ADC0 conversion!  
    DAC0_Management(); // Manage DAC0 conversion!  
    *  
    *  
    *  
    LED_Panel();      // Manage 8*8 LED Panel!  
}
```

(Super Loop)

Varje “uppgift”  
blir ett ”eget”  
program med en  
egen Super loop!

```
int main(void)  
{  
    xTaskCreate(perfMonitor, "task1", 512, (void*)NULL, 2, NULL);  
    xTaskCreate(LCD_WR_Queue, "task2", 512, (void*)NULL, 2, NULL);  
    xTaskCreate(U0_TX_Queue, "task3", 512, (void*)NULL, 2, NULL);  
    *  
    *  
    *  
    xTaskCreate(LED_Panel, "task12", 512, (void*)NULL, 2, NULL);  
  
    vTaskStartScheduler();  
    // Should not reach here...  
  
    while(1); // ...to be sure.  
}
```



# Features



- Pre-emptive or co-operative operation
- Very flexible task priority assignment
- Flexible, fast and light weight task notification mechanism
- Queues
- Binary semaphores
- Counting semaphores
- Mutexes
- Recursive Mutexes
- Software timers
- Event groups
- Tick hook functions
- Idle hook functions
- Stack overflow checking
- Trace recording
- Task run-time statistics gathering



# Distribution



## FreeRTOS

- Source Directory containing the FreeRTOS source files
- Demo Directory containing pre-configured and port specific FreeRTOS demo projects

### -Source

- tasks.c FreeRTOS source file - always required
- list.c FreeRTOS source file - always required
- queue.c FreeRTOS source file - nearly always required
- timers.c FreeRTOS source file - optional
- event\_groups.c FreeRTOS source file - optional
- croutine.c FreeRTOS source file - optional

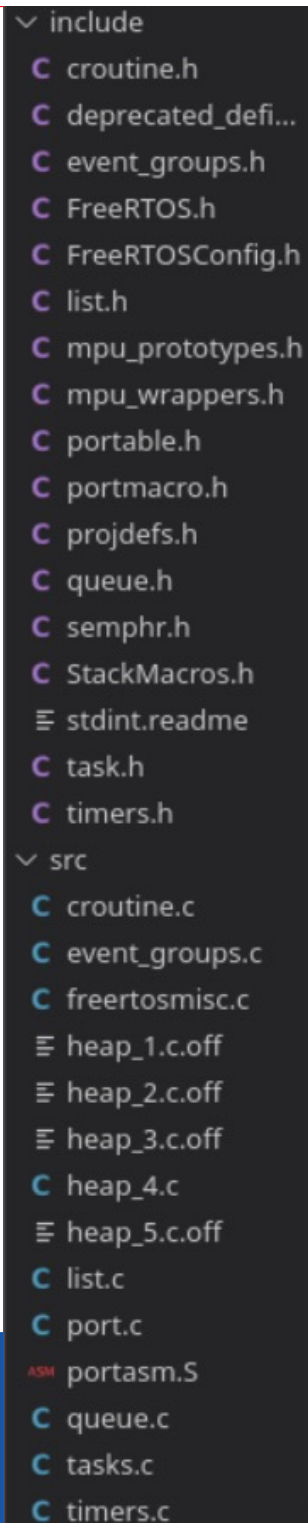
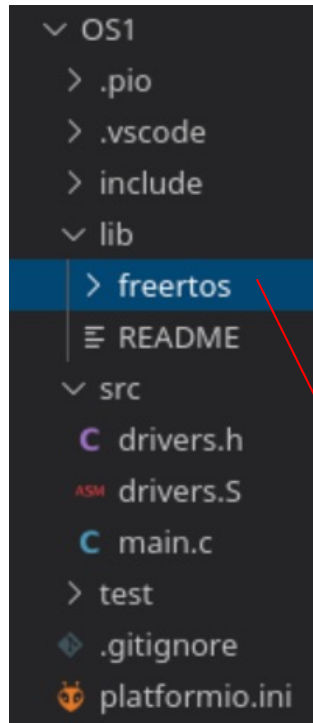
includes

.h include files

### -portable Directory containing all port specific source files

- MemMang Directory containing the 5 alternative heap allocation source files
- [compiler 1] Directory containing port files specific to compiler 1
  - [architecture 1] Contains files for the compiler 1 architecture 1 port
  - [architecture 2] Contains files for the compiler 1 architecture 2 port
  - [architecture 3] Contains files for the compiler 1 architecture 3 port
- [compiler 2] Directory containing port files specific to compiler 2
  - [architecture 1] Contains files for the compiler 2 architecture 1 port
  - [architecture 2] Contains files for the compiler 2 architecture 2 port
  - [etc.]



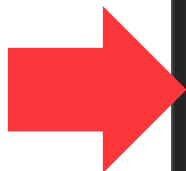


Alla filer till vänster, utom drivers & main, tillhör FreeRTOS och utgör os:et. Den koden ska inte röras, men måste finnas med i projektet, all koppling sker via ett fåtal väl specificerade api-anrop! I Canvas finns foldern "freertos" som ni bara laddar ner och placerar i projektets folder "lib", så kommer det att fungera!

```
1  #include "FreeRTOS.h" /* Must come first. */
2  #include "task.h"      /* RTOS task related API prototypes. */
3
4  #include "gd32vf103.h" // Main MCU API header
5  #include "drivers.h"   // LED Panel & Keyboard driver API
6
7  int main(void)
8  {
9      // Perform any driver setup necessary...
10
11     // Create (initially) needed application tasks...
12
13     // Start the Scheduler!
14     vTaskStartScheduler();
15     // Should not reach here...
16
17     while(1); // ...to be sure!
18 }
```

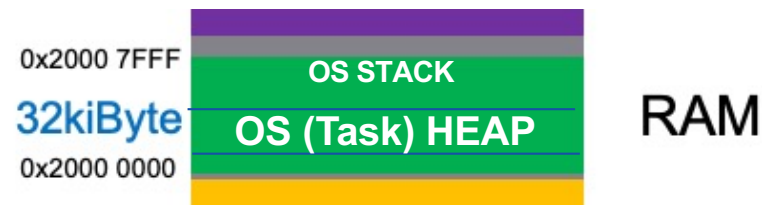
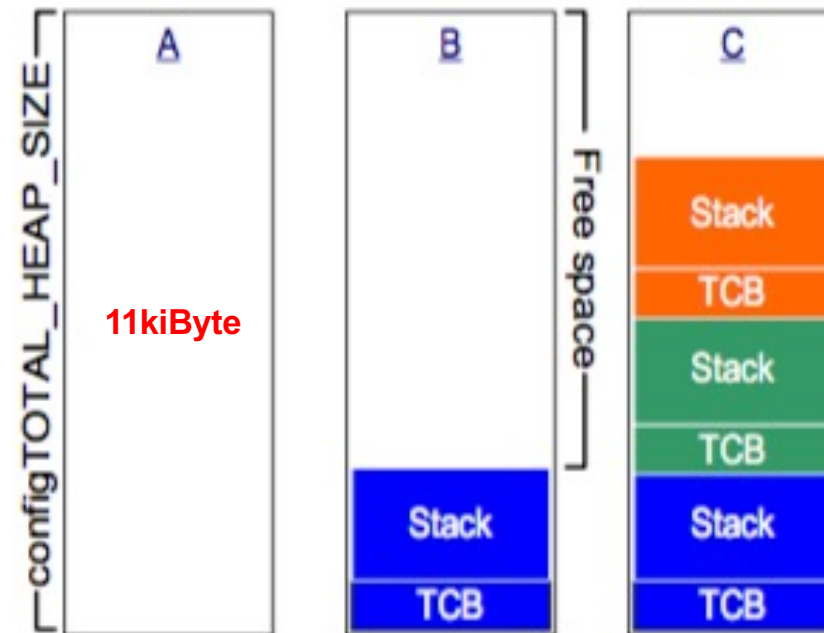


# Memory Allocation...



```
include
C croutine.h
C deprecated_defi...
C event_groups.h
C FreeRTOS.h
C FreeRTOSConfig.h
C list.h
C mpu_prototypes.h
C mpu_wrappers.h
C portable.h
C portmacro.h
C projdefs.h
C queue.h
C semphr.h
C StackMacros.h
stdint.readme
C task.h
C timers.h
src
C croutine.c
C event_groups.c
C freertosmisc.c
heap_1.c.off
heap_2.c.off
heap_3.c.off
C heap_4.c
heap_5.c.off
list.c
```

configTOTAL\_HEAP\_SIZE



- heap\_1 - the very simplest, does not permit memory to be freed.
- heap\_2 - permits memory to be freed, but does not coalesce adjacent free blocks.
- heap\_3 - simply wraps the standard malloc() and free() for thread safety.
- heap\_4 - coalescences adjacent free blocks to avoid fragmentation. Includes absolute address placement option.
- heap\_5 - as per heap\_4, with the ability to span the heap across multiple non-adjacent memory areas.

First-fit

# Task Management...

Ett TCB är 112 Byte

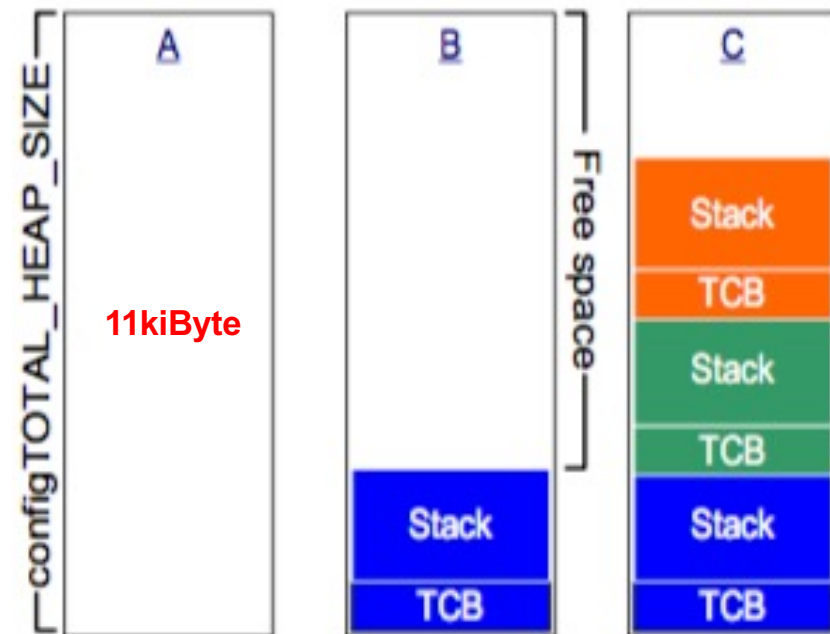
Stack 2048 Byte\*

M.a.o 2160 Byte/task!

Teoretiskt

$11 \cdot 1024 / 2160 = 5$  task!

(\*Ska räcka till "alla" funktioners stack frames för den "längsta" anropskedjan i programmet + context switch data!)

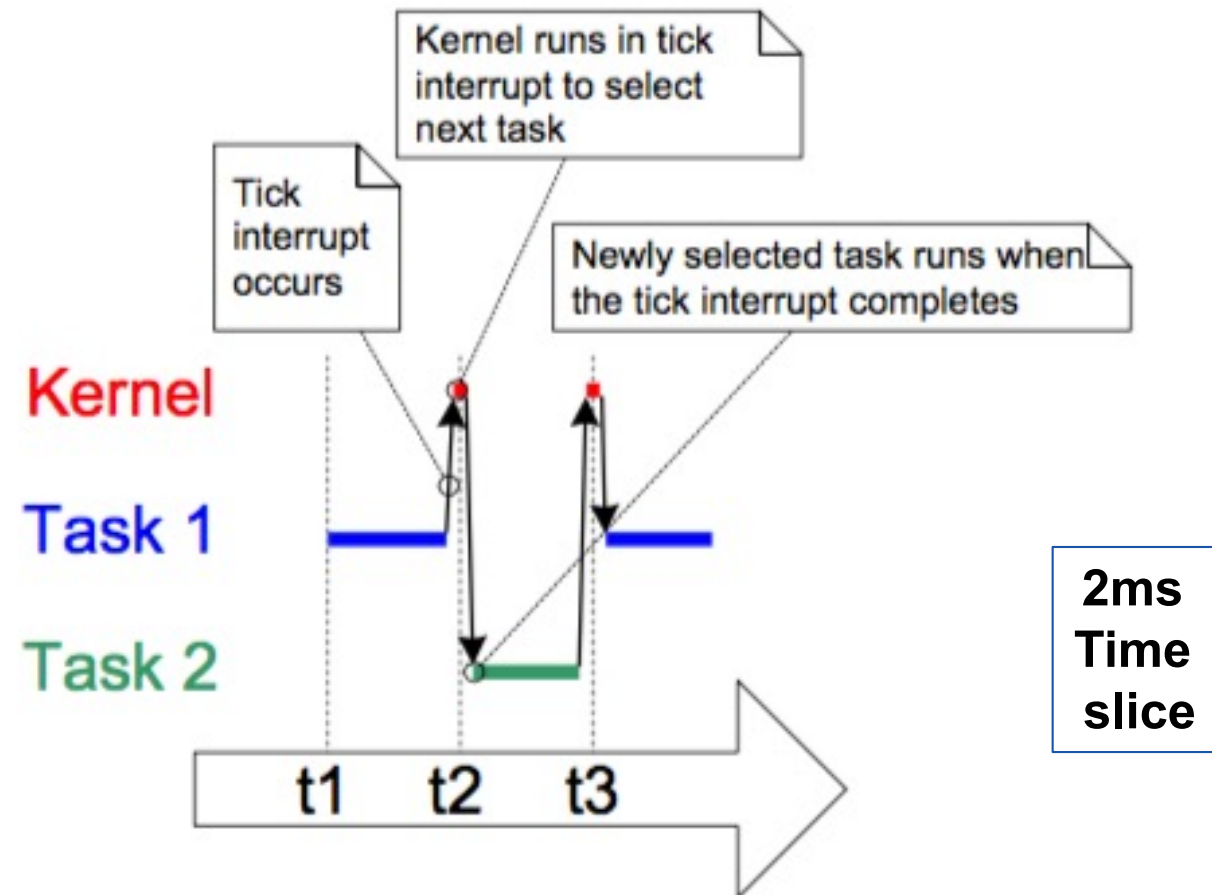


```
void aTaskFunction( void *pvParameters ); (never end)
```

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,           //code
    const char * const pcName,           //name
    uint16_t usStackDepth,               //stack
    void *pvParameters,                  //param
    UBaseType_t uxPriority,               //prio
    TaskHandle_t *pxCreatedTask );       //Id
```



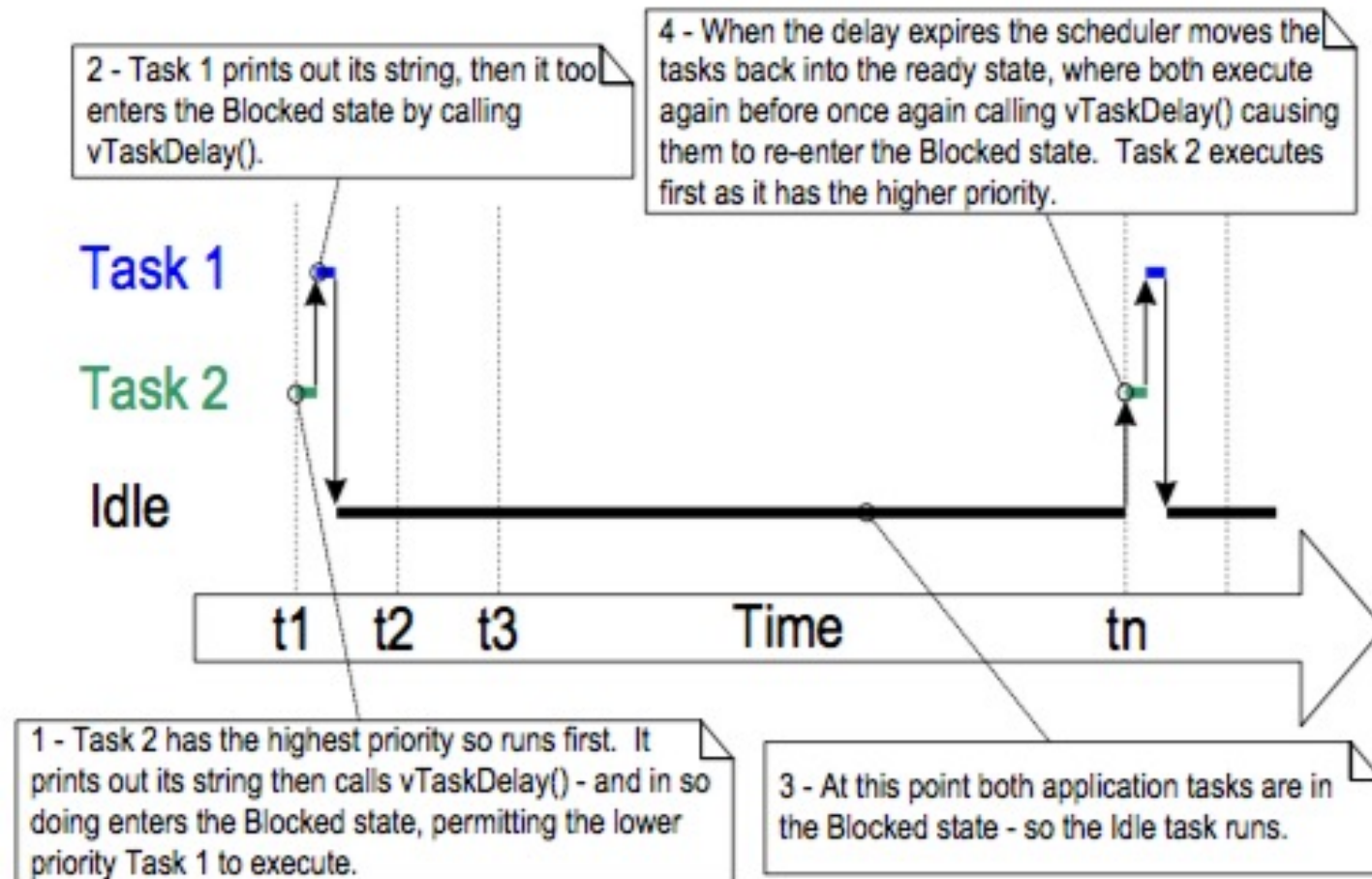
# Task Management...



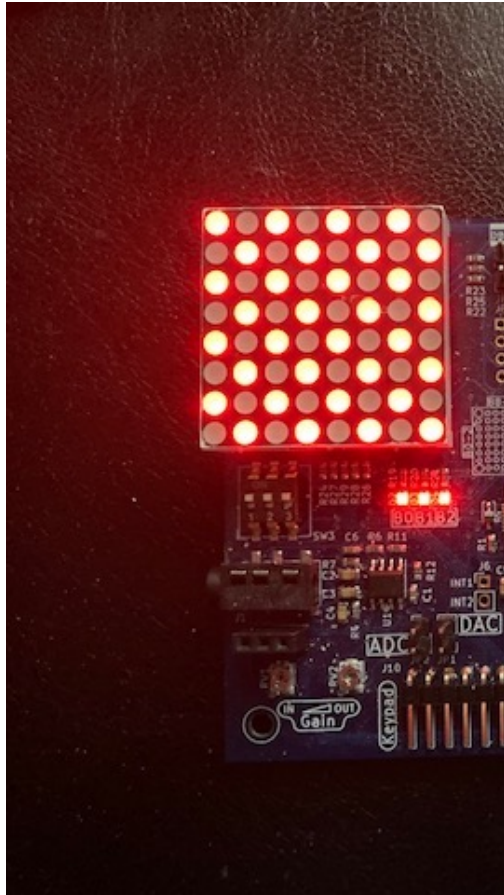
vTaskDelay(ticks)  
vTaskDelayUntil(tick)



# Task Management...



void vApplicationIdleHook(void) = egen idle funktion =  
Låg prio uppgifter/Monitorera Heap/Low Power Mode...



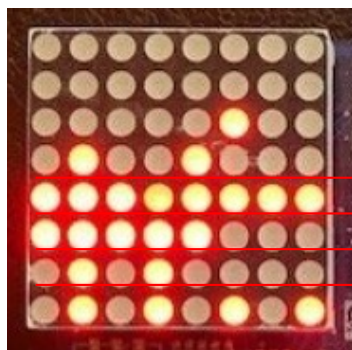
```
1  #include "FreeRTOS.h" /* Must come first. */
2  #include "task.h"      /* RTOS task related API prototypes. */
3
4  #include "gd32vf103.h" // Main MCU API header
5  #include "drivers.h"   // LED Panel & Keyboard driver API
6
7  void ledPanel(void *pvParameters){      // The Led Panel Task:
8      while (1) {                        // Led Panel Super Loop...
9          l88row(colset());               // ...show next row...
10         vTaskDelay(1);                  // ...wait 2ms and repeat!
11     }                                    // (Must not terminate)
12 }
13
14 int main(void)
15 {
16     // Perform any driver setup necessary...
17     colinit();                          // Initialize column toolbox
18     l88init();                           // Initialize 8*8 led toolbox
19
20     // Create (initially) needed application tasks...
21     xTaskCreate(ledPanel, "8*8LED", 512,(void*)NULL, 2, NULL);
22
23     // Start the Scheduler!
24     vTaskStartScheduler();
25     // Should not reach here...
26
27     while(1); // ...to be sure!
28 }
```

Back in business...





xPortGetFreeHeapSize()



0x00000448

"ledTask1"

"ledTask2"

"ledTask3"

"Räknar upp" värdet i raden med  
1 varje sekund, var annan sekund  
och var 3:e sekund

## Lab #1

```
8 void ledPanel(void *pvParameters){ // The Led Panel Task:
9     volatile char data[1800];
10    while (1) { // Led Panel Super Loop...
11        l88row(colset()); // ...show next row...
12        vTaskDelay(1); // ...wait 2ms and repeat!
13
14        l88mem(0,
15        l88mem(1,
16        l88mem(2,
17        l88mem(3,
18    } // (Must not terminate)
19 }
20
21 void ledTask(void *pvParameters){ // The Led Panel Task:
22     // ...show next row...
23     // Led Panel Super Loop...
24     // ...wait 2+id ms and repeat!
25     // (Must not terminate)
26 }
27
28
29 int main(void) {
30     int id[4]={0,1,2,3}; // "TIPS"
31
32     // Perform any driver setup necessary...
33     colinit(); // Initialize column toolbox
34     l88init(); // Initialize 8*8 led toolbox
35
36     // Create (initially) needed application tasks...
37     xTaskCreate(ledPanel, "LED", 512, (void*)NULL, 2, NULL);
38     xTaskCreate(ledTask,
39     xTaskCreate(ledTask,
40     xTaskCreate(ledTask,
41
42     // Start the Scheduler!
43     vTaskStartScheduler();
44     // Should not reach here...
```



**Queues, Semaphores...**

**Queues...**

**Semaphores...**

**Timers...**

**Critical Sections...**

**Events & Notifications...**

**Mer om dylikt i block #3 & #4...**



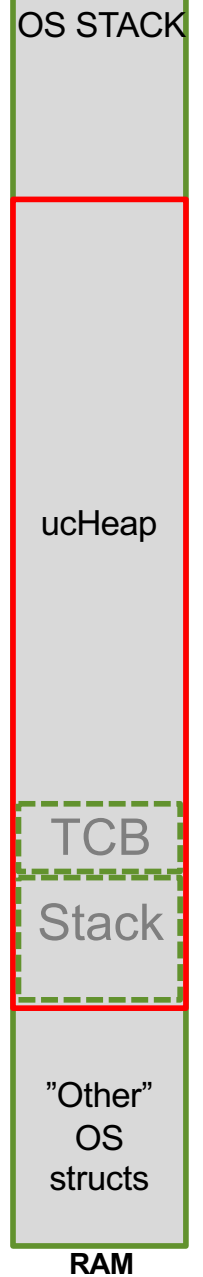
# Vad händer när “programmet” startar...

```
68 #define configUSE_PREEMPTION 1
69 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
70 #define configUSE_TICKLESS_IDLE 0
71 #define configCPU_CLOCK_HZ ((uint32_t)SystemCoreClock)
72 #define configRTC_CLOCK_HZ ((uint32_t)TIMER_FREQ)
73 #define configTICK_RATE_HZ ( ( TickType_t ) 500 )
74 #define configMAX_PRIORITIES ( 4 ) //0 - 3 共6等级, idle独
75 #define configMINIMAL_STACK_SIZE ( ( unsigned short ) 128 )
76 #define configMAX_TASK_NAME_LEN 16
77 #define configUSE_16_BIT_TICKS 0
78 #define configIDLE_SHOULD_YIELD 0
79 #define configUSE_TASK_NOTIFICATIONS 1
80 #define configUSE_MUTEXES 1
81 #define configUSE_RECURSIVE_MUTEXES 0
82 #define configUSE_COUNTING_SEMAPHORES 1
83 #define configQUEUE_REGISTRY_SIZE 10
84 #define configUSE_QUEUE_SETS 0
85 #define configUSE_TIME_SLICING 1
86 #define configUSE_NEWLIB_REENTRANT 0
87 #define configENABLE_BACKWARD_COMPATIBILITY 0
88 #define configNUM_THREAD_LOCAL_STORAGE_POINTERS 5
89
90 /* Memory allocation related definitions. */
91 #define configSUPPORT_STATIC_ALLOCATION 0
92 #define configSUPPORT_DYNAMIC_ALLOCATION 1
93 #define configTOTAL_HEAP_SIZE 11*1024
94 #define configAPPLICATION_ALLOCATED_HEAP 0
```

FreeRTOSConfig.h

```
100 /* Allocate the memory for the heap. */
101 #if( configAPPLICATION_ALLOCATED_HEAP == 1 )
102     /* The application writer has already defined the array used for the RTOS
103     heap - probably so it can be placed in a special segment or address. */
104     extern uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
105 #else
106     static uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
107 #endif /* configAPPLICATION_ALLOCATED_HEAP */
```

Heap\_4.c





```
676 BaseType_t xTaskCreate( TaskFunction_t pxTaskCode,
677                         const char * const pcName,
678                         const uint16_t usStackDepth,
679                         void * const pvParameters,
680                         UBaseType_t uxPriority,
681                         TaskHandle_t * const pxCreatedTask ) /*lint !e971 Unqualified char types are allowed for
682 {
683     TCB_t *pxNewTCB;
```

```
1  #include "FreeRTOS.h"
2  #include "task.h"
3
4  #include "ledPanel.h"
5  #include "l88.h"
6
7  void ledPanelTask( void *pvParameters )
8  {
9      while( 1 )
10         vTaskDelay( 100 );
11 }
12
13
14 int main(void)
15 {
16     // Perform any driver setup necessary...
17     colinit();           // Initialize column toolbox
18     l88init();           // Initialize 8*8 led toolbox
19
20     // Create (initially) needed application tasks...
21     xTaskCreate(ledPanelTask, "8*8LED", 512, (void*)NULL, 2, NULL);
22
23     // Start the Scheduler!
24     vTaskStartScheduler();
25     // Should not reach here...
26
27     while(1); // ...to be sure!
28 }
```

```
713 StackType_t *pxStack;
714
715 /* Allocate space for the stack used by the task being created. */
716 pxStack = ( StackType_t * ) pvPortMalloc( ( ( ( size_t ) usStackDepth ) * sizeof( StackType_t ) ) ); /*lint !e971
717
718 if( pxStack != NULL )
719 {
720     /* Allocate space for the TCB. */
721     pxNewTCB = ( TCB_t * ) pvPortMalloc( sizeof( TCB_t ) ); /*lint !e961 MISRA exception as the casts are
722
723 if( pxNewTCB != NULL )
724 {
725     /* Store the stack location in the TCB. */
726     pxNewTCB->pxStack = pxStack;
727 }
```

tasks.c

```
293 typedef struct tskTaskControlBlock
294 {
295     volatile StackType_t *pxTopOfStack; /*< Points to the top of the stack.
296
297     #if ( portUSING_MPU_WRAPPERS == 1 )
298         xMPU_SETTINGS xMPUSettings; /*< The MPU settings for the task.
299     #endif
300
301     ListItem_t xStateListItem; /*< The list that the task is in.
302     ListItem_t xEventListItem; /*< Used to reference the task in an event list.
303     UBaseType_t uxPriority; /*< The priority of the task.
304     StackType_t *pxStack; /*< Points to the stack.
305     char pcTaskName[ configMAX_TASK_NAME_LEN ]; /*< The task name.
```

OS STACK

ucHeap

TCB 2  
8\*8LED

Stack  
8\*8LED

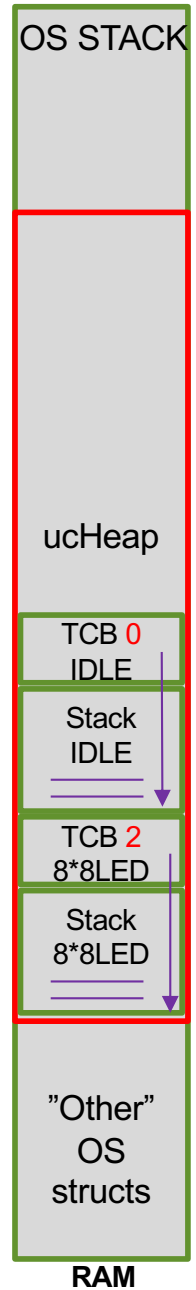
"Other"  
OS  
structs

RAM





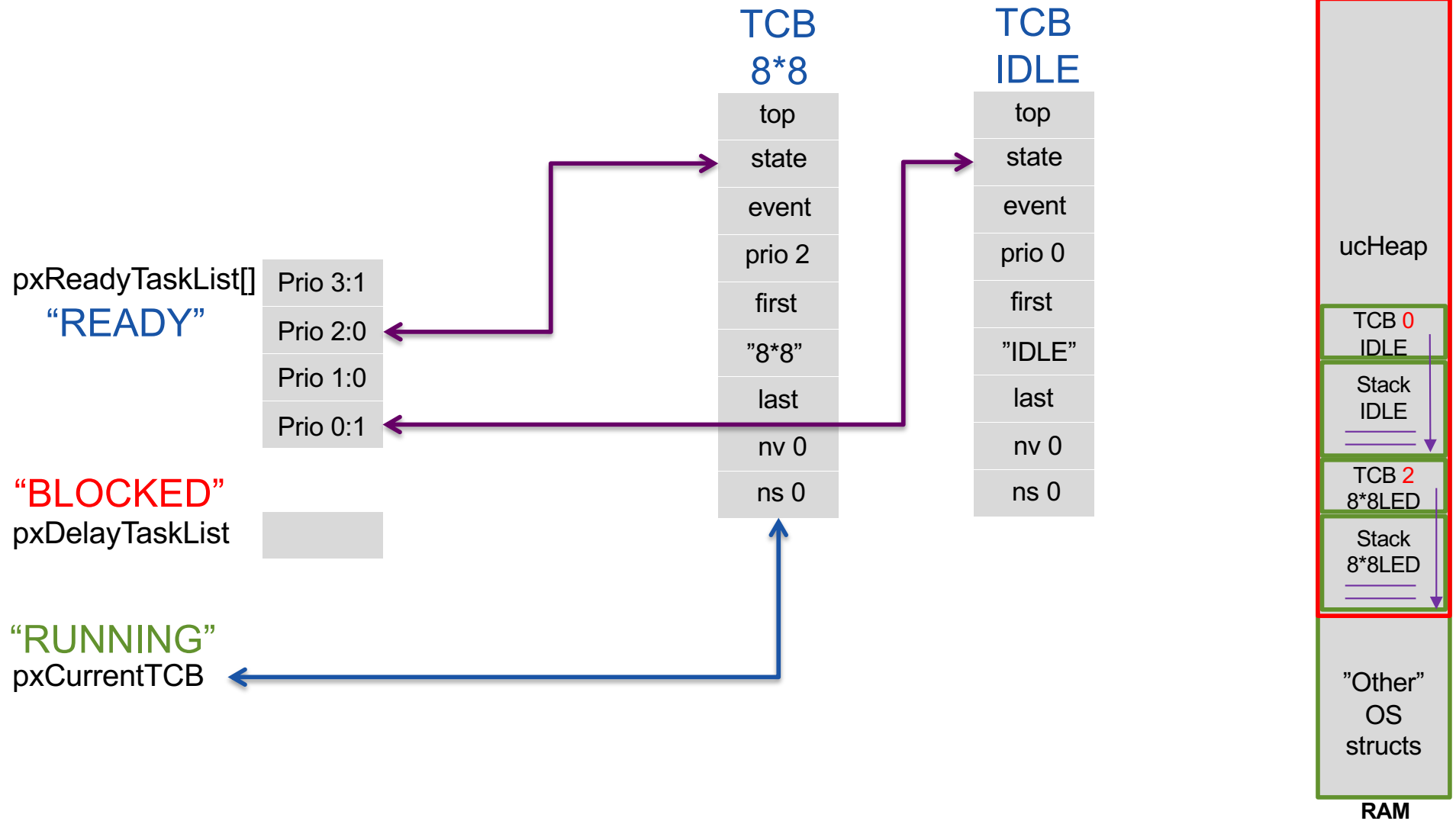
```
1821 void vTaskStartScheduler( void )
1822 {
1823     BaseType_t xReturn;
1824
1825     /* Add the idle task at the lowest priority. */
1826     #if( configSUPPORT_STATIC_ALLOCATION == 1 )
1827     {
1828         StaticTask_t *pxIdleTaskTCBBuffer = NULL;
1829         StackType_t *pxIdleTaskStackBuffer = NULL;
1830         uint32_t ulIdleTaskStackSize;
1831
1832         /* The Idle task is created using user provided RAM - obtain the
1833          address of the RAM then create the idle task. */
1834         vApplicationGetIdleTaskMemory( &pxIdleTaskTCBBuffer, &pxIdleTaskStackBuf
1835         xIdleTaskHandle = xTaskCreateStatic(     prvIdleTask,
1836                                               "IDLE",
1837                                               ulIdleTaskStackSize,
1838                                               ( void * ) NULL,
1839                                               ( tskIDLE_PRIORITY | portPRIVILE
1840                                               pxIdleTaskStackBuffer,
1841                                               pxIdleTaskTCBBuffer ); /*lint le
1842
1843         if( xIdleTaskHandle != NULL )
1844         {
1845             xReturn = pdPASS;
1846         }
1847         else
1848         {
1849             xReturn = pdFAIL;
1850         }
1851     }
1852     #else
1853     {
1854         /* The Idle task is being created using dynamically allocated RAM. */
1855         xReturn = xTaskCreate(     prvIdleTask,
1856                               "IDLE", configMINIMAL_STACK_SIZE,
1857                               ( void * ) NULL,
1858                               ( tskIDLE_PRIORITY | portPRIVILEGE_BIT ),
1859                               &xIdleTaskHandle ); /*lint le961 MISRA exception
1860     }
1861     #endif /* configSUPPORT_STATIC_ALLOCATION */
1862
1863     #if ( configUSE_TIMERS == 1 )
1864     {
1865         if( xReturn == pdPASS )
1866         {
1867             xReturn = xTimerCreateTimerTask();
1868         }
1869     }
```







...till slut får 8\*8LED "köra" (bare metal)



...men hur länge då?

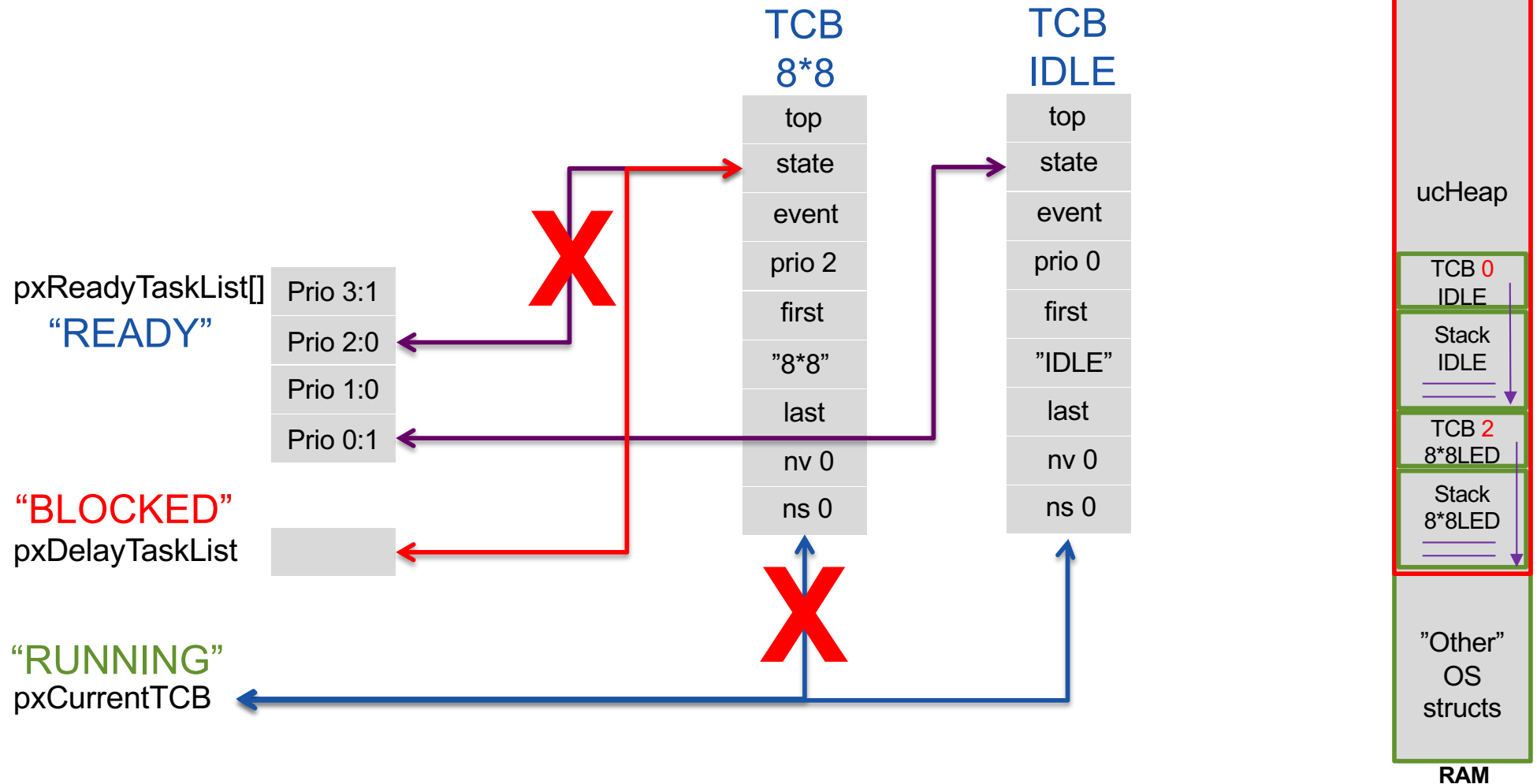
# Context Switch

```
237  /* Macro for restoring task context */
238  .macro popREGFILE
239
240      /* Restore registers,
241      Skip global pointer because that does not change */
242      LOAD    x1, 1 * REGBYTES(sp)
243      //LOAD  x3, 3 * REGBYTES(sp)
244      LOAD    x4, 4 * REGBYTES(sp)
245      LOAD    x5, 5 * REGBYTES(sp)
246      LOAD    x6, 6 * REGBYTES(sp)
247      LOAD    x7, 7 * REGBYTES(sp)
248      LOAD    x8, 8 * REGBYTES(sp)
249      LOAD    x9, 9 * REGBYTES(sp)
250      LOAD    x10, 10 * REGBYTES(sp)
251      LOAD    x11, 11 * REGBYTES(sp)
252      LOAD    x12, 12 * REGBYTES(sp)
253      LOAD    x13, 13 * REGBYTES(sp)
254      LOAD    x14, 14 * REGBYTES(sp)
255      LOAD    x15, 15 * REGBYTES(sp)
256  #ifndef __riscv_32e
257      LOAD    x16, 16 * REGBYTES(sp)
258      LOAD    x17, 17 * REGBYTES(sp)
259      LOAD    x18, 18 * REGBYTES(sp)
260      LOAD    x19, 19 * REGBYTES(sp)
261      LOAD    x20, 20 * REGBYTES(sp)
262      LOAD    x21, 21 * REGBYTES(sp)
263      LOAD    x22, 22 * REGBYTES(sp)
264      LOAD    x23, 23 * REGBYTES(sp)
265      LOAD    x24, 24 * REGBYTES(sp)
266      LOAD    x25, 25 * REGBYTES(sp)
267      LOAD    x26, 26 * REGBYTES(sp)
268      LOAD    x27, 27 * REGBYTES(sp)
269      LOAD    x28, 28 * REGBYTES(sp)
270      LOAD    x29, 29 * REGBYTES(sp)
271      LOAD    x30, 30 * REGBYTES(sp)
272      LOAD    x31, 31 * REGBYTES(sp)
273  #endif
274      addi    sp, sp, REGBYTES * 36
275  .endm
```

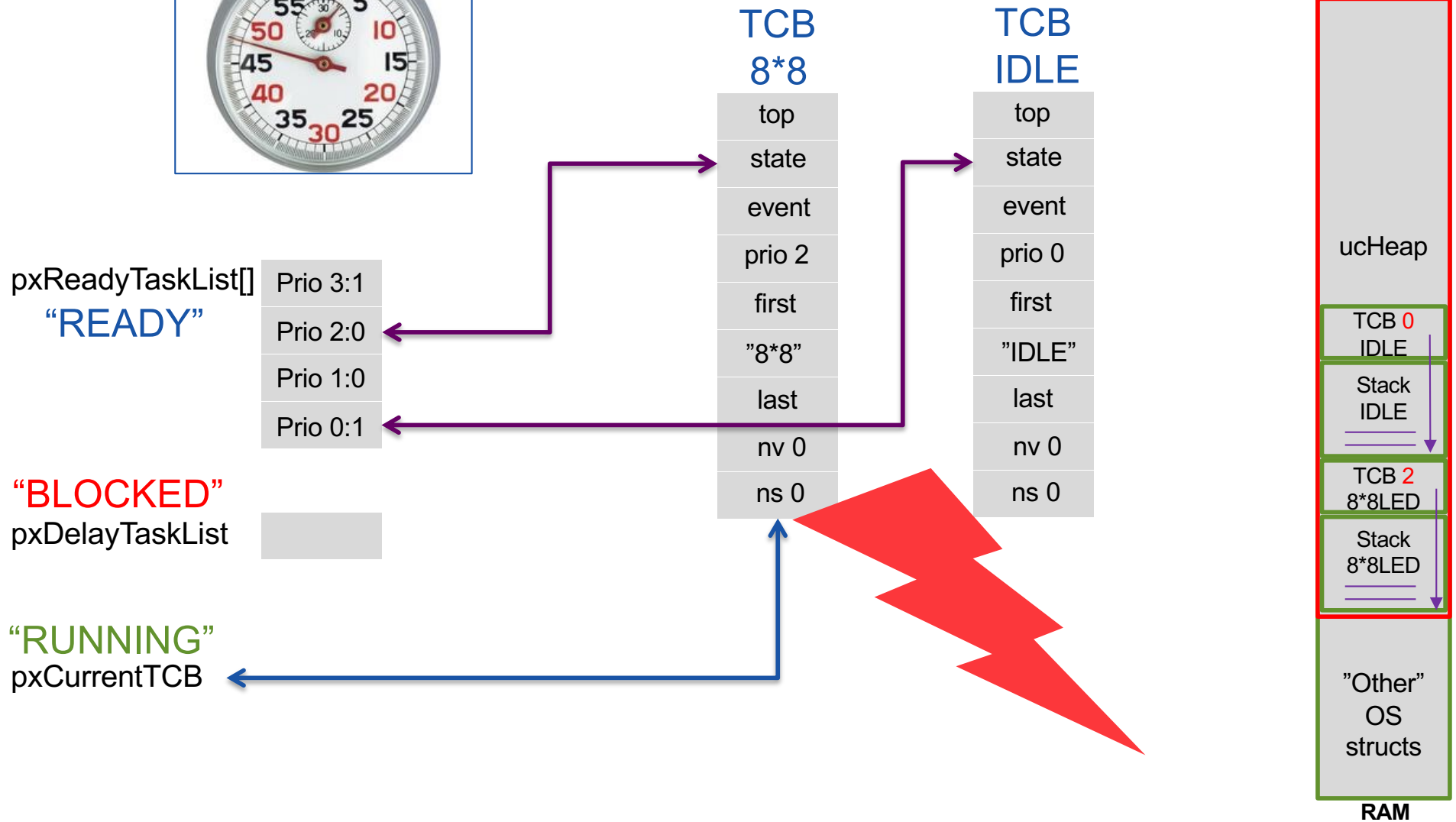
```
319  /*restore mstatus/mepc and tcb */
320  .macro portRESTORE_CONTEXT_EXCP
321      .global pxCurrentTCB
322      /* Load stack pointer from the current TCB */
323      LOAD    sp, pxCurrentTCB
324      LOAD    sp, 0x0(sp)
325      /* Load task program counter */
326      LOAD    t0, 34*REGBYTES(sp)
327      csrw    CSR_MSUBM, t0
328      LOAD    t0, 33*REGBYTES(sp)
329      csrw    CSR_MEPC, t0
330      LOAD    t0, 32*REGBYTES(sp)
331      csrw    CSR_MSTATUS, t0
332  .endm
```



```
7 void ledPanel(void *pvParameters){ // The Led Panel Task:
8     while (1) { // Led Panel Super Loop...
9         l88row(colset()); // ...show next row...
10        vTaskDelay(1); // ...wait 2ms and repeat!
11    } // (Must not terminate)
12 }
```



...8\*8 är BLOCKED, så IDLE-processen får köra, för evigt?



...efter 2ms sker ett “tick” – OS:et återtar kontrollen, 8\*8 är inte längre BLOCKED utan schemaläggs, & allt börjar om!



# Begrepp

**Throughput vs. Fairness**

**Turnaround time**

**First Come, First Served**

**Shortest Job First**

**Shortest time to completion first**

**Response time**

**Round Robin**

**Multi-level Feedback Queue**

**Proportional Share**

**Stride Scheduling**

**Hard vs. Soft Real Time System**

**Rate Monotonic Scheduling**

**Earliest Deadline First**

**Idle Task**

**Register & Volatile**

**Bit-wise & shift**

**Struct + union & at**

